

Einsendeaufgabe Softwaretechnik

TST E1

4. Semester SoSe26

Eingereicht von: Marvin Bock, 106334

Kurs: Softwaretechnik (BHT MIB 20 S26)

Kursleitung: Prof. Dr.-Ing. Stefan Edlich

Aufgabe 1 JUnit

In Aufgabe 1 wurde Eclipse verwendet. Ich habe das Beispiel Taschenrechner verwendet, um den Test mit der Exception durchzuführen. Die JUnit-Tests wurde bereits in anderen Modulen angewendet, in der Aufgabe Buildmanagement können Sie dies nachvollziehen.

```
1 package taschenrechner;
2
3 import static org.junit.jupiter.api.Assertions.*;
4 import org.junit.jupiter.api.*;
5
6
7 class taschenrechner_test {
8     Taschenrechner c;
9
10
11     @BeforeEach
12     void setUp() throws Exception {
13         c = new Taschenrechner();
14     }
15
16     @Test
17     void testDiv() {
18         assertEquals(c.div(10, 5), 2);
19     }
20
21     @Test
22     void testDivDurchNull() {
23         assertThrows(
24             IllegalArgumentException.class, () -> c.div(10, 0)
25         );
26     }
27 }
28
29 }
```

Abbildung 1: Klasse taschenrechner_test

```
1 package taschenrechner;
2
3 class Taschenrechner {
4
5     public int div(int a, int b) {
6
7         //if (b == 0) {
8             //throw new IllegalArgumentException("Division durch 0 nicht erlaubt.");
9         // }
10
11         return a/b;
12     }
13 }
14 }
```

Abbildung 2: Klasse Taschenrechner, Exception auskommentiert

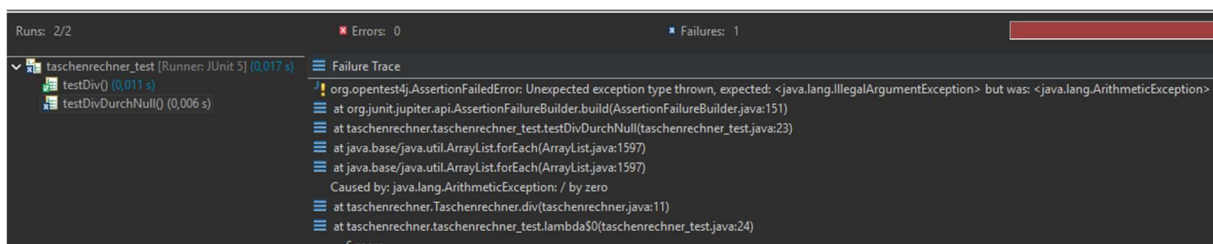
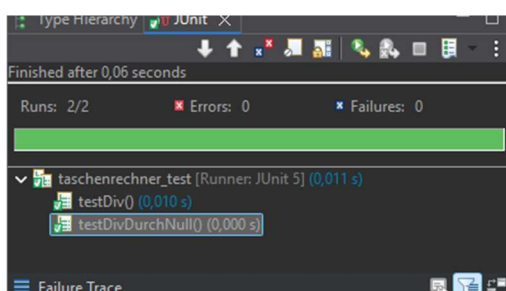


Abbildung 1: Ausführung der TestKlasse

Java selbst wirft eine ArithmeticException.

Nach Einkommentierung ist die Exception korrekt und der Test erfolgreich.



Aufgabe 2 ShoppingCart:

Der Code befindet sich im REPO, es wurden Insgesamt 12 Commits gemacht. Commit 11 ist doppelt.

Den Aufbau des Codes fand ich als schwierig, da es keine Definitionen oder Vorgaben in der Aufgabenstellung gibt. Z.B. Soll eine Liste oder eine Map genutzt werden, welche Daten sollen verarbeitet werden, INT oder double. Ich habe mich letztendlich für eine Map und Integer entschieden. Kleinere Syntax Schwierigkeiten habe ich nachgeschlagen.

Aufgabe 4 Mutation Testing:

Ich habe das Plugin in Eclipse verwendet, damit es funktioniert musste ich jedoch den Compiler-Level auf 17 heruntersetzen, da es scheinbar keine aktuelle Java Unterstützung vom PIT-Plugin gibt. Die Fehlersuche war zeitraubend.

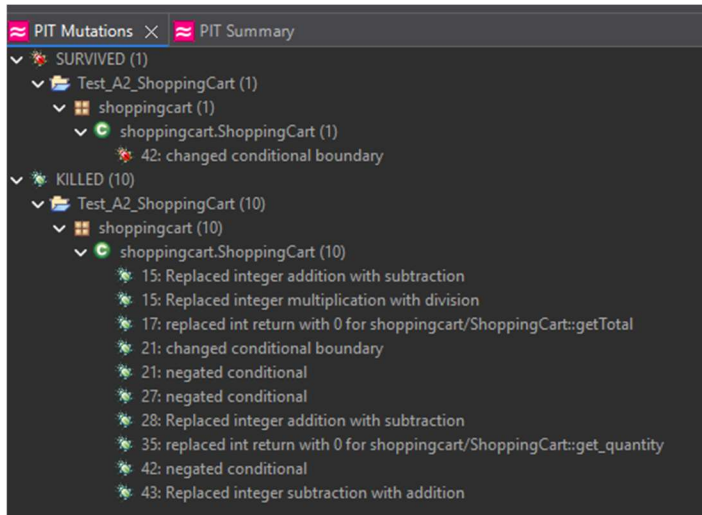


Abbildung 2: PIT Mutations

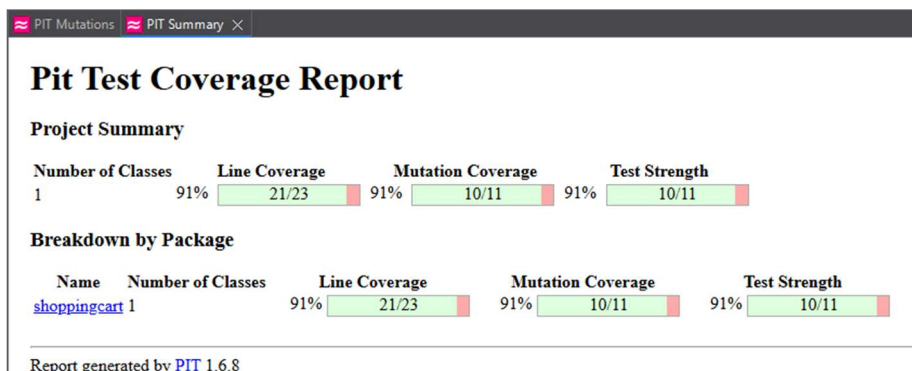


Abbildung 4: PIT Summary

Im Code wurde bei Zeile 42 die Bedingung geändert. Hierbei gibt es eine Variation, welche nicht durch die Tests korrekt abgefangen wird. Anstelle den Artikel zu löschen wird die Menge auf 0 reduziert. Der Artikel verbleibt in der Map mit der Menge 0. Mit dem nachfolgenden Test, konnte ich die Mutation killen.

```
38 public void remove_item(String name) {  
39  
40     int menge = warenmenge.get(name);  
41  
42     if (menge > 1) {  
43         warenmenge.put(name, menge - 1);  
44     } else {  
45         warenmenge.remove(name);  
46         warenkorb.remove(name);  
47     }  
48 }
```

Abbildung 3: Code Zeile 42

```
74 @Test  
75 void test_remove_item_Mutation() {  
76     ShoppingCart cart = new ShoppingCart();  
77     cart.addItem("VierItem", 4);  
78     cart.remove_item("VierItem");  
79  
80     assertFalse(cart.warenmenge.containsKey("VierItem"));  
81  
82     System.out.println("Test remove Mutation" + cart.warenkorb);  
83     System.out.println("Test remove Mutation" + cart.warenmenge);  
84 }
```

Da die Bearbeitungszeit der Aufgabe bereits weit überschritten war, habe ich nach einer ersten eigenen Analyse die KI-Unterstützung bei der Deutung von PIT Mutation eingesetzt: Der anschließende Mutation-Test war erfolgreich mit 11(all) Killed.

Prompt:

Ich habe in meinem Java Projekt das Eclipse Plugin PIT rüberlaufen lassen und es wurde eine Mutation festgestellt in der Klasse ShoppingCart Zeile 42. mit der Info „changed conditional boundary“. Hilfe mir den Fehler zu interpretieren und mit einem Assert abzufangen, damit die Mutation nicht überlebt.
Hier mein Code
Class: package shoppingcart;

```
import java.util.HashMap;  
import java.util.Map;
```

Mehr anzeigen ▾

Gut, jetzt können wir das sauber wie bei einem echten Mutation-Test auseinandernehmen.

Ausgabe:

Gut, jetzt können wir das sauber wie bei einem echten Mutation-Test auseinandernehmen.

1. Was bedeutet „changed conditional boundary“?

Diese Mutation verändert eine **Grenzbedingung** in einer if-Abfrage.

Typisch ist z. B.:

Dein Code:

```
</> Java  
if (menge > 1)
```

PIT-Mutation:

```
</> Java  
if (menge >= 1)
```

oder

```
</> Java  
if (menge < 1)
```

2. Wo ist die betroffene Stelle bei dir?

In deiner Klasse:

```
</> Java  
if (menge > 1) {  
    warenmenge_out(name, menge - 1);  
}
```

+ Stelle irgendeine Frage

📌 2. Wo ist die betroffene Stelle bei dir?

In deiner Klasse:

```
</> Java
if (menge > 1) {
    warenmenge.put(name, menge - 1);
} else {
    warenmenge.remove(name);
    warenkorb.remove(name);
}
```

👉 genau diese Zeile ist Zeile 42 laut PIT

⚠️ 3. Was ist das Problem logisch?

Deine Logik bedeutet:

Menge	Verhalten
2+	reduzieren
1	löschen

PIT verändert z. B. zu:

```
</> Java
if (menge >= 1)
```

Dann passiert bei:

```
menge = 1
```

👉 falsches Verhalten:

- wird nicht gelöscht
- sondern auf 0 oder inkonsistent gesetzt

+ Stelle irgendeine Frage

- wird nicht gelöscht
- sondern auf 0 oder inkonsistent gesetzt

! 4. Warum überlebt die Mutation?

Schau deine Tests an:

```
</> Java
assertEquals(0, cart.get_quantity("EinItem"));
```

ODER indirekt:

```
</> Java
assertEquals(10, cart.getTotal());
```

👉 Problem:

Du prüfst NICHT:

Ist der Key wirklich entfernt worden?

Du prüfst nur:

Ist das Ergebnis rechnerisch 0?

✨ 5. Der fehlende „Kill-Test“

Du brauchst einen Test, der das **echte Entfernen erzwingt**:

✓ Lösung 1 (beste Variante für PIT)

```
</> Java
@Test
void test_item_is_removed_completely() {
    ShoppingCart cart = new ShoppingCart();
```

+ Stelle irgendeine Frage


```
@Test
void test_item_is_removed_completely() {
    ShoppingCart cart = new ShoppingCart();

    cart.addItem("EinItem", 1);

    cart.remove_item("EinItem");

    assertFalse(cart.warenmenge.containsKey("EinItem"));
    assertFalse(cart.warenkorb.containsKey("EinItem"));
}
```

✓ Lösung 2 (wenn du Encapsulation behalten willst)

Dann brauchst du eine Methode:

```
Java

public boolean containsItem(String name) {
    return warenmenge.containsKey(name);
}
```

Test:

```
Java

assertFalse(cart.containsItem("EinItem"));
```

6. Warum genau dieser Assert die Mutation „killed“

Weil PIT bei Mutation testet:

Original

Mutation

remove bei 1

bleibt m ↓ der Key bestehen

+ Stelle irgendeine Frage



Original	Mutation
remove bei 1	bleibt mit 0 oder Key bestehen

👉 Dein neuer Test erkennt:

- Key ist noch da ❌
- obwohl er weg sein sollte

 **7. Was du in deiner Abgabe schreiben kannst**

Du kannst genau das schreiben:

Die Mutation „changed conditional boundary“ veränderte die Bedingung `menge > 1` in eine alternative Grenzbedingung. Dadurch würde ein Element bei Menge 1 nicht korrekt entfernt werden. Dieser Fehler konnte durch einen zusätzlichen Test, der überprüft, ob der Eintrag vollständig aus der HashMap entfernt wurde, erfolgreich erkannt und die Mutation „getötet“ werden.

 **8. Kurzfazit**

👉 Dein Problem ist NICHT der Code

👉 sondern dein Test prüft nur Werte, nicht Struktur

Wenn du willst, kann ich dir jetzt noch zeigen:

- ✓ wie du 100% PIT Coverage bekommst
- ✓ oder wie du deine Tests „perfekt bewertbar“ formulierst
- ✓ oder wie du das sauber für die Abgabe dokumentierst

📄 👍 💬 ↗️ ↺️ ⋮

+ Stelle irgendeine Frage

